

International Islamic University Islamabad

Faculty of Engineering and Technology

Department of Electrical and Computer Engineering



AI & Machine Learning Lab

Experiment No. 3: Performing Data Analysis and Visualizing Results Using Pandas and Matplotlib

Name of Student: [Bushra Nazir](#).....

Registration No.: [1071-F22](#).....

Date of Experiment: [10-3-2026](#).....

Submitted To: [Engr.Asad](#).....

Objectives:

- To develop the ability to create and customize graphical representations (line plots, scatter plots, bar charts, histograms, pie charts, and subplots) using Matplotlib for effective data interpretation.
- To analyze and visualize statistical relationships in datasets using Seaborn, including heatmaps and correlation plots for machine learning applications.
- To interpret visual patterns, trends, and feature relationships to support data analysis, anomaly detection, and model evaluation in machine learning.

Equipment & Tools Required

Hardware:

- Personal Computer/Laptop with a minimum of 4GB RAM (8GB recommended)

Software & Tools:

- Jupyter Notebook (via Anaconda) or Google Colab
- Internet Access (*for downloading packages and using Google Colab if needed*)

Pandas

Pandas is a powerful open-source Python library designed for data manipulation and analysis. Built on top of NumPy, it provides high-performance, easy-to-use data structures such as Series (one-dimensional) and DataFrame (two-dimensional) that simplify handling structured data. Pandas is particularly well-suited for tasks like data cleaning, filtering, transformation, and aggregation, making it a cornerstone tool in data science and analytics workflows. Its intuitive syntax and rich set of functions allow for seamless integration with other scientific libraries, enabling efficient exploratory data analysis and preparation for machine learning or statistical modeling.

Introduction to Data Visualization

Why Visualization?

Data visualization is a crucial step in data analysis and machine learning. It helps to:

- Identify patterns, trends, and anomalies in data.
- Summarize large datasets in an intuitive manner.

- Improve decision-making by making insights more understandable.
- Communicate findings effectively through graphical representations

For Machine Learning (ML), data visualization helps in:

- Understanding feature distributions
- Identifying outliers
- Exploring relationships between variables
- Evaluating model performance

Installing Matplotlib and Seaborn

To get started with data visualization, we need to install Matplotlib and Seaborn.

Installation

Use the following command to install these libraries:

```
pip install matplotlib seaborn
```

Importing Libraries

```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

Basic Matplotlib Plots

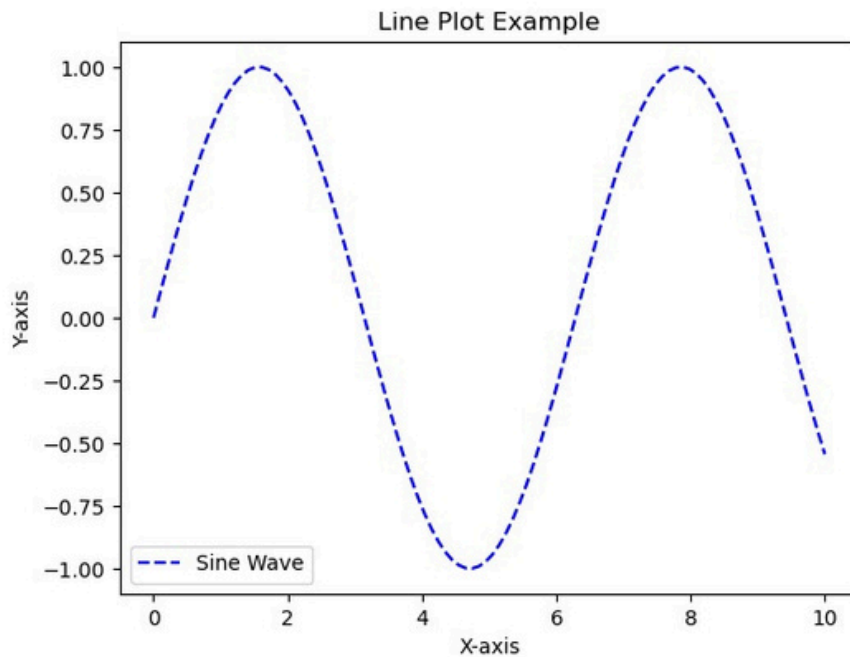
Matplotlib allows us to create a variety of plots for data visualization.

1. Line Plot (plt.plot())

A line plot is used to visualize trends over time.

```
x = np.linspace(0, 10, 100)
y = np.sin(x)

plt.plot(x, y, label="Sine Wave", color="blue", linestyle="--")
plt.xlabel("X-axis")
plt.ylabel("Y-axis")
plt.title("Line Plot Example")
plt.legend()
plt.show()
```

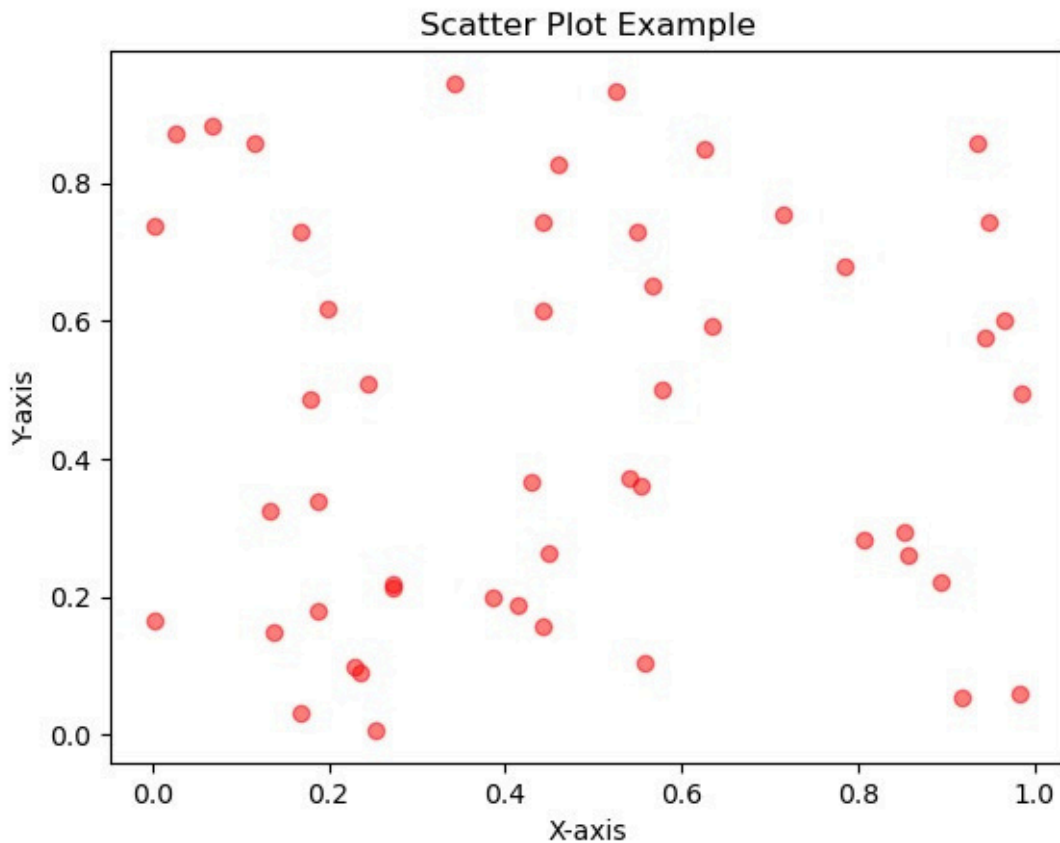


2. Scatter Plot (plt.scatter())

A scatter plot helps visualize relationships between two variables.

```
x = np.random.rand(50)
y = np.random.rand(50)

plt.scatter(x, y, color="red", alpha=0.5, marker="o")
plt.xlabel("X-axis")
plt.ylabel("Y-axis")
plt.title("Scatter Plot Example")
plt.show()
```



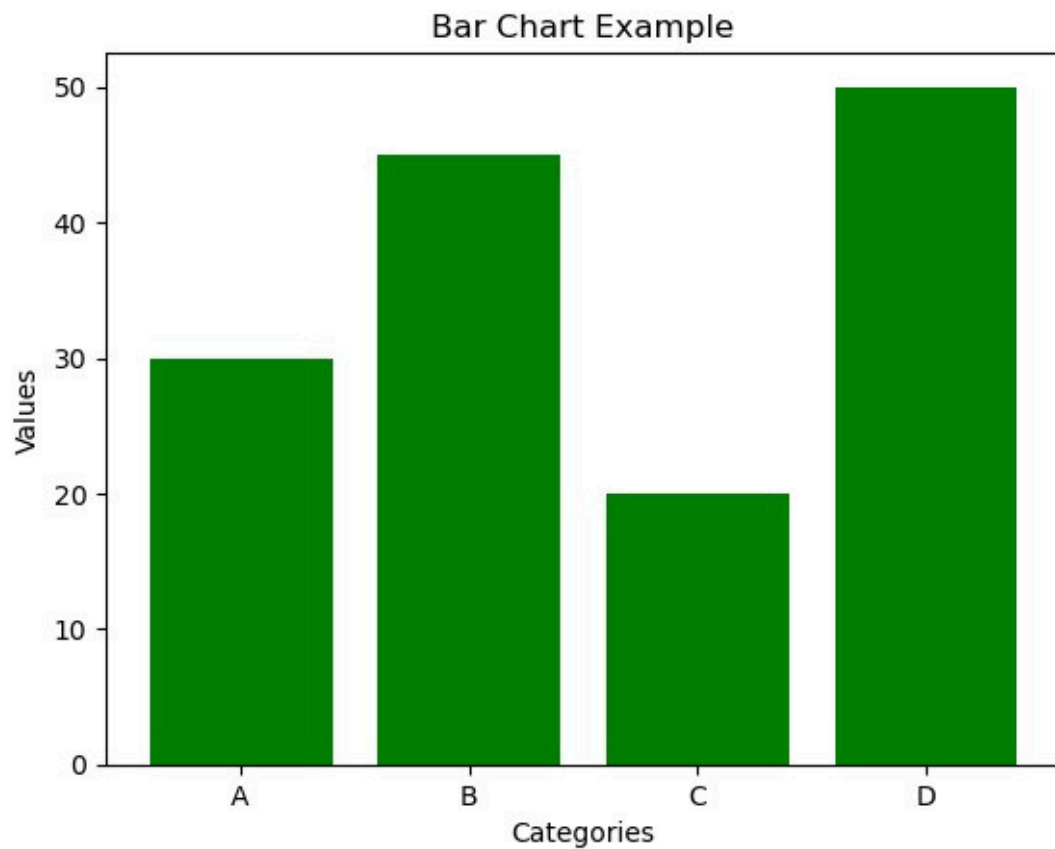
Scattering is Useful for: Identifying correlations and clustering patterns.

3. Bar Chart (plt.bar())

Bar charts are used to compare categorical data.

```
categories = ["A", "B", "C", "D"]
values = [30, 45, 20, 50]

plt.bar(categories, values, color="green")
plt.xlabel("Categories")
plt.ylabel("Values")
plt.title("Bar Chart Example")
plt.show()
```

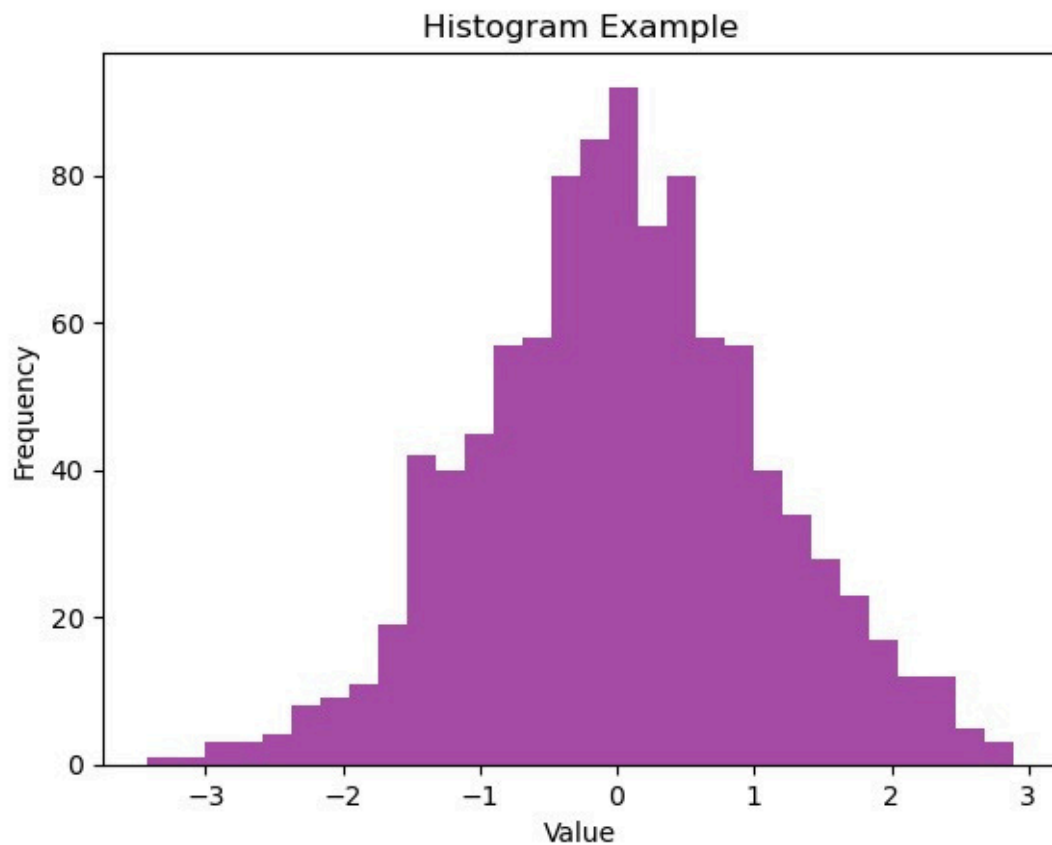


4. Histogram (plt.hist())

A histogram is used to visualize the distribution of a dataset.

```
data = np.random.randn(1000)

plt.hist(data, bins=30, color="purple", alpha=0.7)
plt.xlabel("Value")
plt.ylabel("Frequency")
plt.title("Histogram Example")
plt.show()
```



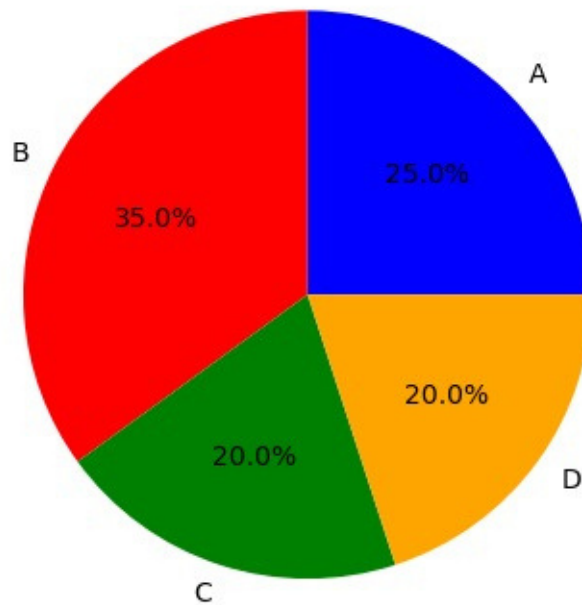
5. Pie Chart (plt.pie())

Pie charts are used to show proportions in a dataset.

```
labels = ["A", "B", "C", "D"]
sizes = [25, 35, 20, 20]

plt.pie(sizes, labels=labels, autopct="%1.1f%%", colors=["blue", "red", "green", "orange"])
plt.title("Pie Chart Example")
plt.show()
```

Pie Chart Example

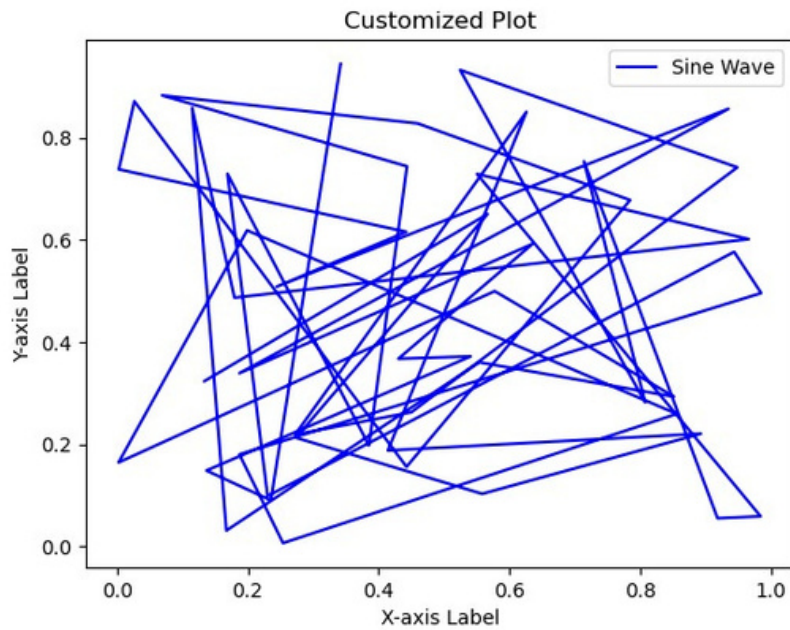


Customizing Plots

Matplotlib allows various customizations to make plots more readable.

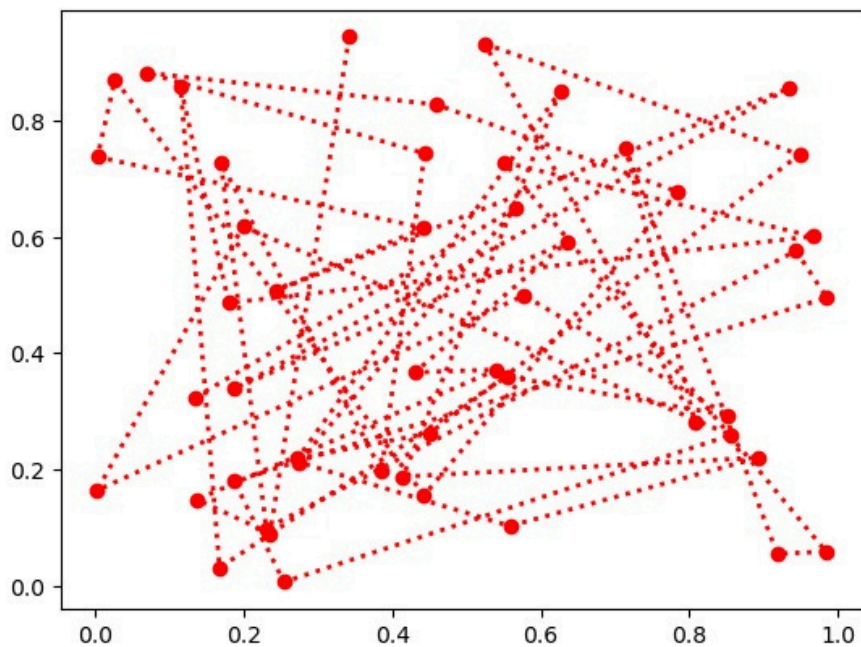
1. Labels, Titles, and Legends

```
plt.plot(x, y, label="Sine Wave", color="blue")
plt.xlabel("X-axis Label")
plt.ylabel("Y-axis Label")
plt.title("Customized Plot")
plt.legend() # Adds a Legend
plt.show()
```

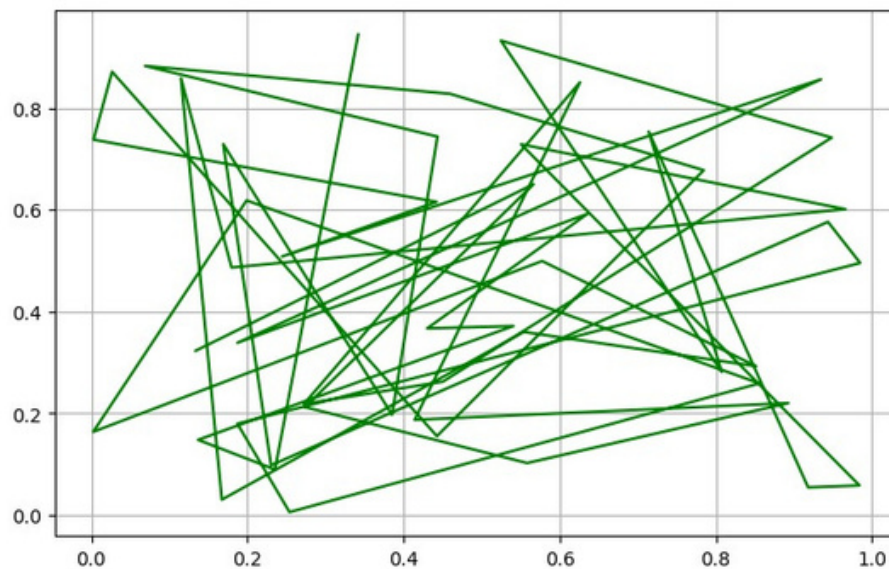
2. Changing Colors and Line Styles

```
plt.plot(x, y, color="red", linestyle="dotted", linewidth=2, marker="o")  
plt.show()
```



3. Adding Grid and Changing Figure Size

```
plt.figure(figsize=(8, 5)) # Set figure size
plt.plot(x, y, color="green")
plt.grid(True) # Add grid
plt.show()
```



Creating Subplots (plt.subplot() and plt.subplots())

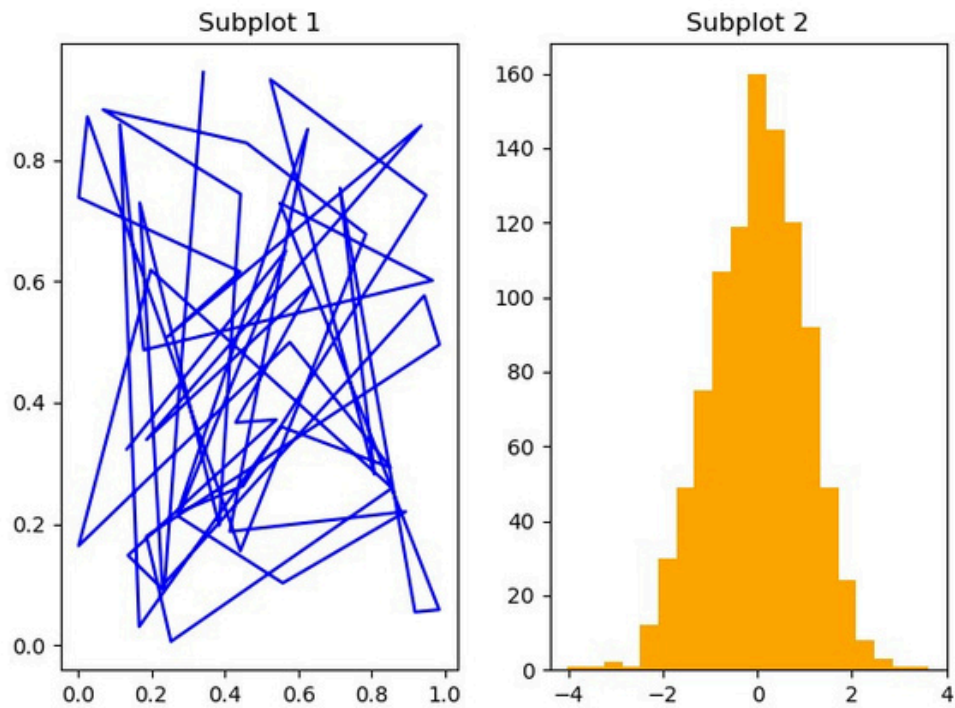
Subplots allow multiple plots in a single figure.

1. Using plt.subplot()

```
plt.subplot(1, 2, 1) # (rows, columns, plot number)
plt.plot(x, y, color="blue")
plt.title("Subplot 1")

plt.subplot(1, 2, 2)
plt.hist(data, bins=20, color="orange")
plt.title("Subplot 2")

plt.tight_layout() # Adjust spacing
plt.show()
```



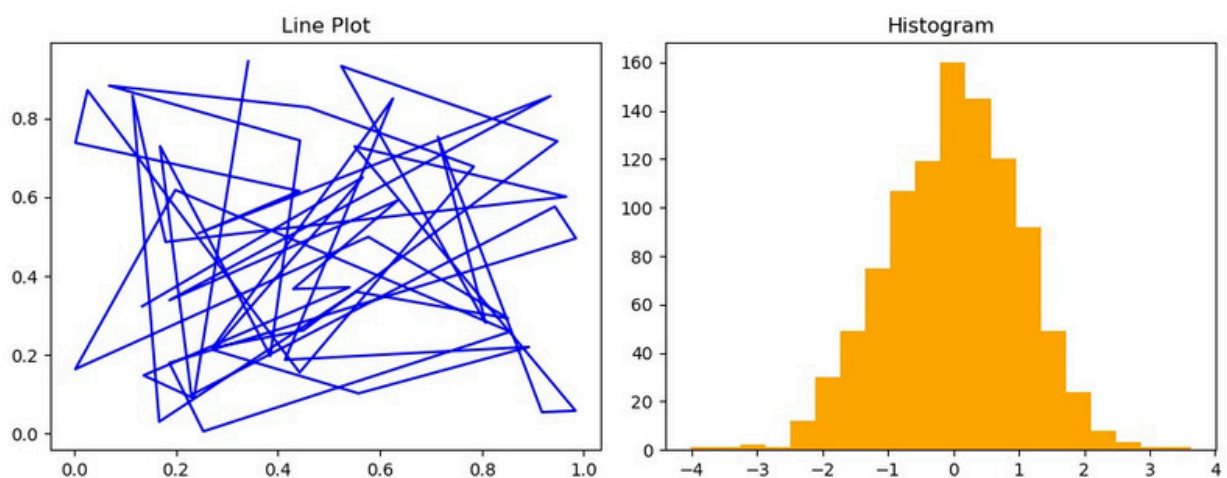
2. Using plt.subplots() (Recommended)

```
fig, axes = plt.subplots(1, 2, figsize=(10, 4))

axes[0].plot(x, y, color="blue")
axes[0].set_title("Line Plot")

axes[1].hist(data, bins=20, color="orange")
axes[1].set_title("Histogram")

plt.tight_layout()
plt.show()
```



8. Advanced Data Visualization

Using NumPy with Seaborn

Seaborn is an advanced data visualization library built on top of Matplotlib. It provides beautiful and informative statistical graphics with minimal code.

Installing and Importing Seaborn

To install Seaborn, use:

```
pip install seaborn
```

Then, import the required libraries:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

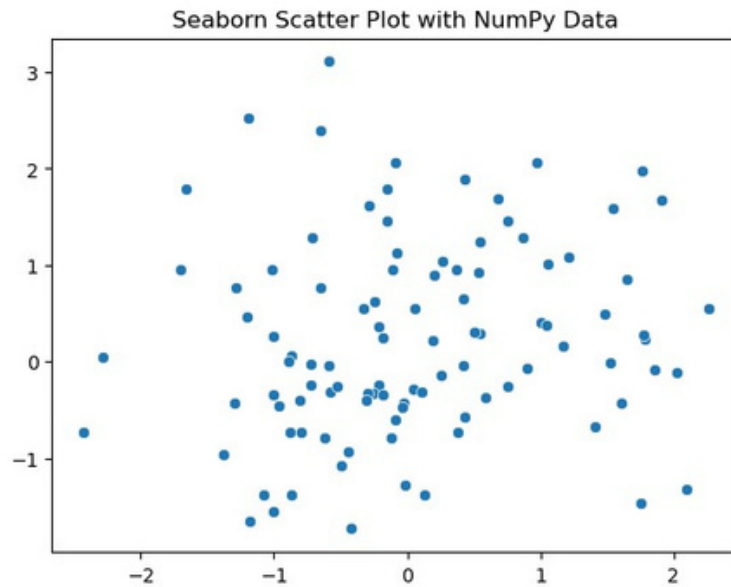
Seaborn works well with NumPy arrays and Pandas DataFrames, making it useful for Machine Learning applications.

Seaborn with NumPy Data

Seaborn can visualize statistical relationships in NumPy-generated data.

```
x = np.random.randn(100)
y = np.random.randn(100)

sns.scatterplot(x=x, y=y)
plt.title("Seaborn Scatter Plot with NumPy Data")
plt.show()
```



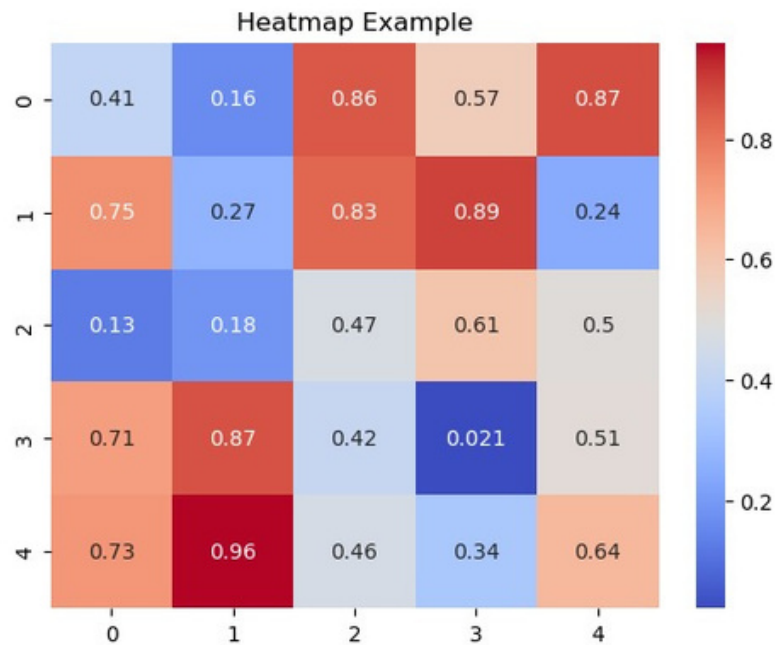
Heatmaps and Correlation Plots

A **heatmap** is useful for visualizing relationships between variables, especially in ML datasets.

Creating a Heatmap

A heatmap displays numerical data using a color gradient.

```
data = np.random.rand(5, 5) # Generate random data
sns.heatmap(data, annot=True, cmap="coolwarm")
plt.title("Heatmap Example")
plt.show()
```

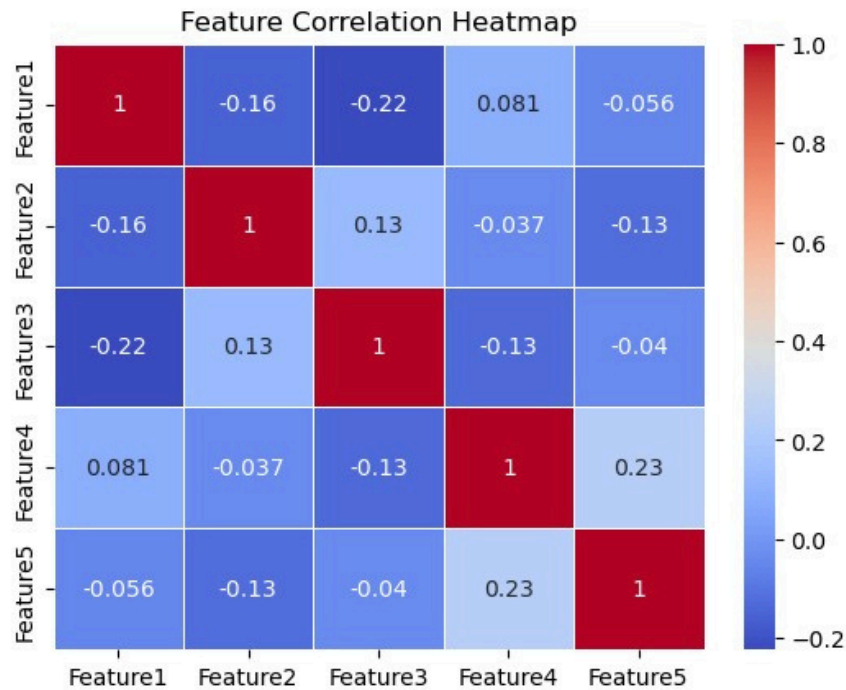
Correlation Plot (Feature Correlations)

In Machine Learning, correlation plots help understand feature relationships.

```
# Creating a dataset with NumPy
np.random.seed(42)
data = np.random.rand(100, 5)
columns = ['Feature1', 'Feature2', 'Feature3', 'Feature4', 'Feature5']
df = pd.DataFrame(data, columns=columns)

# Compute correlation
corr = df.corr()

# Plot heatmap
sns.heatmap(corr, annot=True, cmap="coolwarm", linewidths=0.5)
plt.title("Feature Correlation Heatmap")
plt.show()
```



3D Plotting with Matplotlib (mplot3d)

For 3D visualizations, Matplotlib's mplot3d module is used.

Enabling 3D Plotting

```
from mpl_toolkits.mplot3d import Axes3D

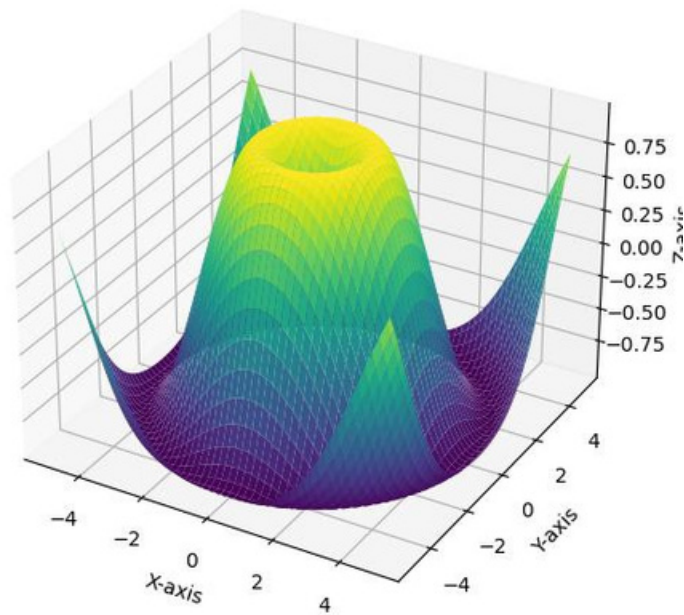
fig = plt.figure(figsize=(8, 6))
ax = fig.add_subplot(111, projection='3d')

# Generating 3D data
x = np.linspace(-5, 5, 100)
y = np.linspace(-5, 5, 100)
X, Y = np.meshgrid(x, y)
Z = np.sin(np.sqrt(X**2 + Y**2))

# Plotting surface
ax.plot_surface(X, Y, Z, cmap="viridis")

ax.set_xlabel("X-axis")
ax.set_ylabel("Y-axis")
ax.set_zlabel("Z-axis")
plt.title("3D Surface Plot")
plt.show()
```

3D Surface Plot



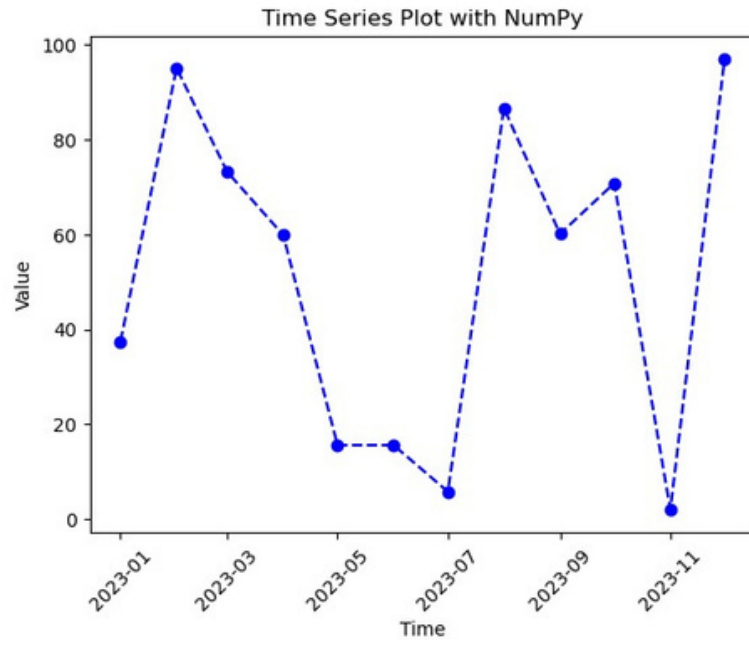
Time Series Visualization

Time series data plays a critical role in ML applications like **weather prediction, and IoT**

stock price forecasting,

```
# Creating a time series with NumPy
time = np.arange('2023-01', '2024-01', dtype='datetime64[M]')
values = np.random.rand(len(time)) * 100

plt.plot(time, values, marker="o", linestyle="--", color="b")
plt.xlabel("Time")
plt.ylabel("Value")
plt.title("Time Series Plot with NumPy")
plt.xticks(rotation=45)
plt.show()
```

EXERCISE

1. Plot a line graph of $y = x^2$ for x ranging from -10 to 10.
 2. Generate a scatter plot using NumPy for:
 - o x values: [1, 2, 3, 4, 5]
 - o y values: [10, 20, 25, 30, 45]
 3. Generate a random NumPy array with 1000 values sampled from a normal distribution.
Use this data to:
 - o Create a histogram to visualize the feature distribution.
 - o Plot a boxplot to detect potential outliers.
 4. Generate a heatmap of feature correlations using NumPy & Seaborn for a random 5×5 dataset.
 5. Implement Softmax Activation Function using NumPy.
 6. Generate a NumPy array with 1000 rows and 5 columns, where each value is randomly sampled from a normal distribution. Display the first 5 rows of the generated data.
-

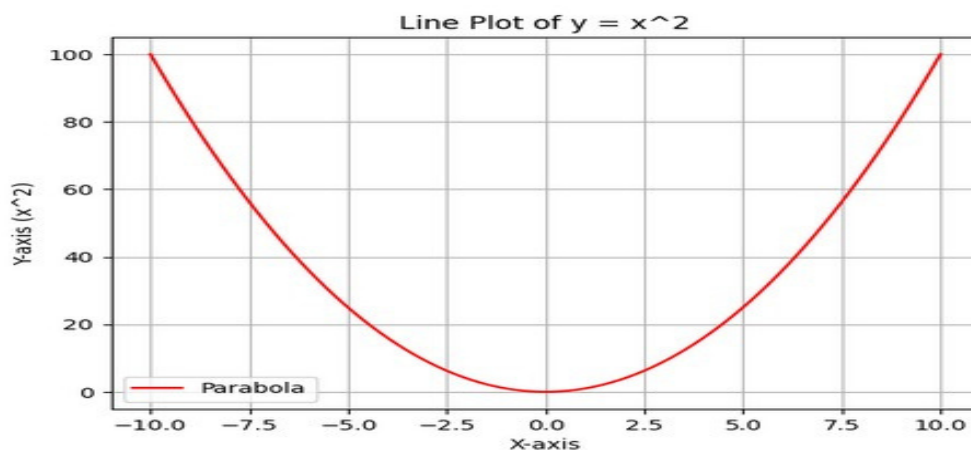
Task 01: Plotting Line Graph

Program:

```
import matplotlib.pyplot as plt
import numpy as np

x = np.linspace(-10, 10, 100)
y = x**2
plt.plot(x, y, label="Parabola", color="red", linestyle="-")
plt.xlabel("X-axis")
plt.ylabel("Y-axis (x^2)")
plt.title("Line Plot of y = x^2")
plt.legend()
plt.grid(True)
plt.show()
```

Output:



Task 02: Generating a scatter plot using a NumPy:

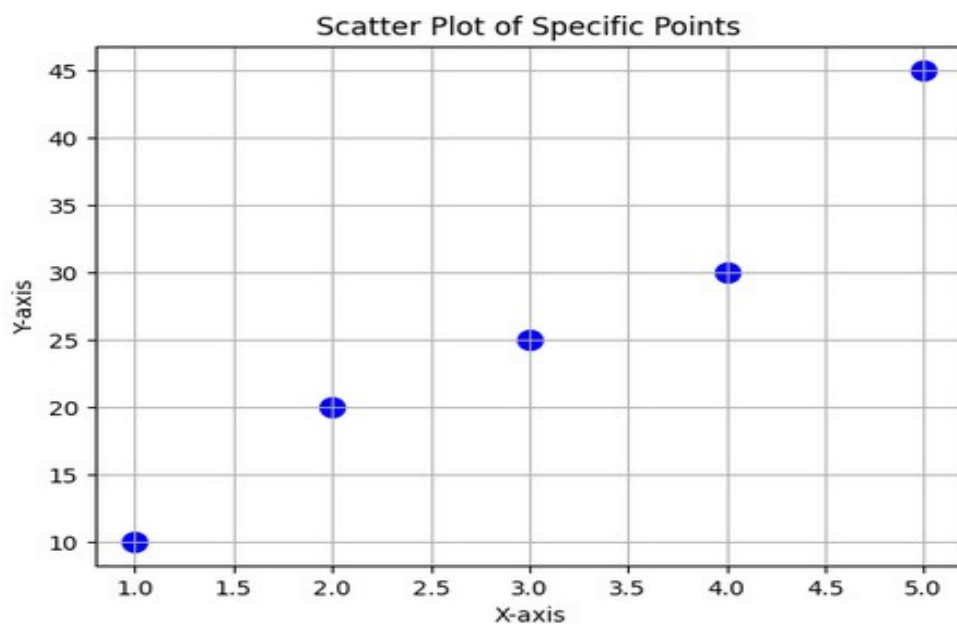
Program:

```
import matplotlib.pyplot as plt
```

```
import numpy as np

x = np.array([1, 2, 3, 4, 5])
y = np.array([10, 20, 25, 30, 45])
plt.scatter(x, y, color="blue", marker="o", s=100)
plt.xlabel("X-axis")
plt.ylabel("Y-axis")
plt.title("Scatter Plot of Specific Points")
plt.grid(True)
plt.show()
```

Output:



Task 03: Generating a random NumPy array with 1000 values sampled from a normal distribution

Program:

```
import matplotlib.pyplot as plt
import numpy as np

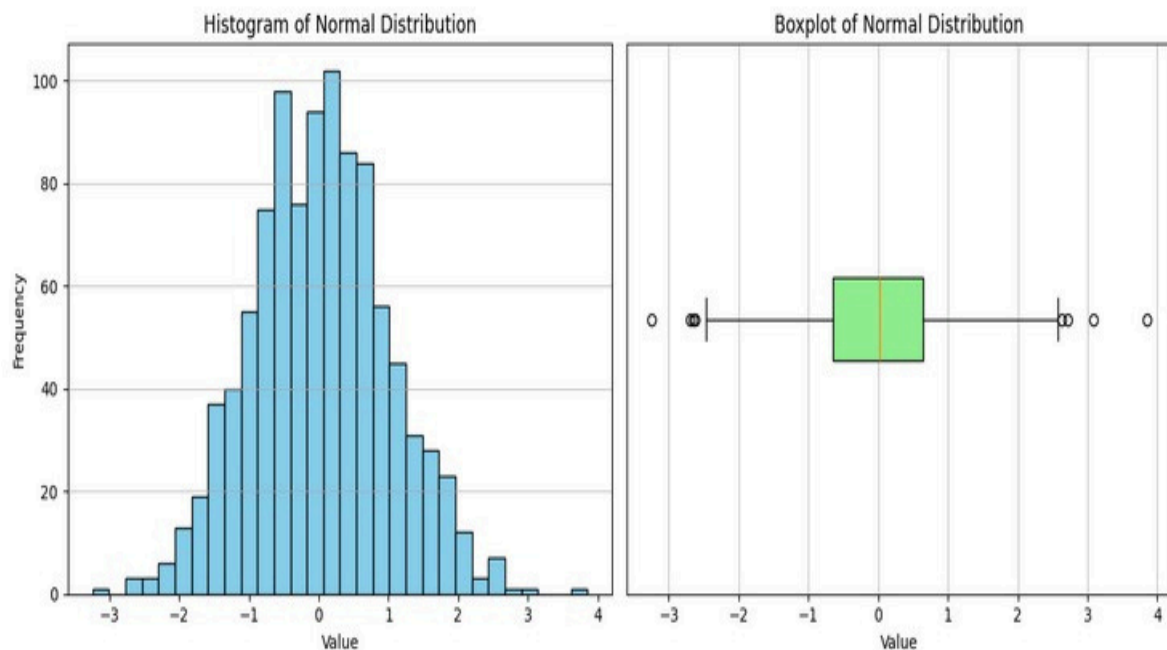
# Generate a random NumPy array with 1000 values from a normal distribution
np.random.seed(42) # for reproducibility
data = np.random.normal(loc=0, scale=1, size=1000)

# Create a histogram to visualize the feature distribution
plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1) # 1 row, 2 columns, first plot
plt.hist(data, bins=30, color='skyblue', edgecolor='black')
plt.title('Histogram of Normal Distribution')
plt.xlabel('Value')
plt.ylabel('Frequency')
plt.grid(axis='y', alpha=0.75)

# Plot a boxplot to detect potential outliers
plt.subplot(1, 2, 2) # 1 row, 2 columns, second plot
plt.boxplot(data, vert=False, patch_artist=True, boxprops=dict(facecolor='lightblue'))
plt.title('Boxplot of Normal Distribution')
plt.xlabel('Value')
plt.yticks([]) # Hide y-axis ticks for a horizontal boxplot
plt.grid(axis='x', alpha=0.75)

plt.tight_layout() # Adjust layout to prevent overlapping titles/labels
plt.show()
```

Output:



Task 04: Generating a heatmap of features correlations using NumPy & Seaborn for a random 5X5 dataset

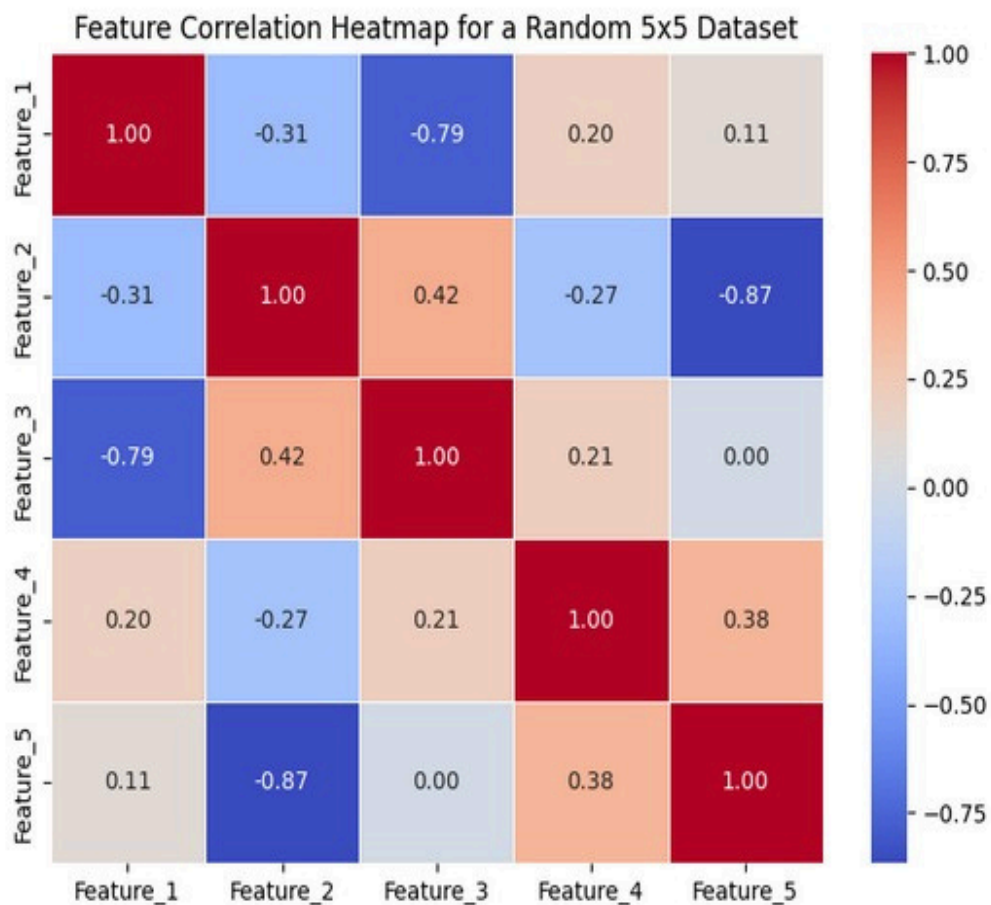
Program:

```
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

np.random.seed(42)
data_5x5 = np.random.rand(5, 5)
df_5x5 = pd.DataFrame(data_5x5, columns=[f'Feature_{i+1}' for i in range(5)])
correlation_matrix = df_5x5.corr()
plt.figure(figsize=(8, 6))
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm',
            fmt=".2f", linewidths=.5)
plt.title('Feature Correlation Heatmap for a Random 5x5 Dataset')
```

```
plt.show()
```

Output:



Task 05: Implementing softmax activation Function using NumPy

Program:

```
import numpy as np
```



```
def softmax(x):
    """
    Compute the softmax activation function for an input array x.
    x: A NumPy array (can be 1D or 2D).
    Returns: A NumPy array with the softmax probabilities.
    """
    e_x = np.exp(x - np.max(x, axis=-1, keepdims=True))
    return e_x / np.sum(e_x, axis=-1, keepdims=True)
print("--- 1D Array Example ---")
arr_1d = np.array([1.0, 2.0, 3.0])
print(f"Input array: {arr_1d}")
print(f"Softmax output: {softmax(arr_1d)}")
print("\n--- 2D Array Example ---")
arr_2d = np.array([[1.0, 2.0, 3.0], [4.0, 5.0, 6.0]])
print(f"Input array:\n{arr_2d}")
print(f"Softmax output:\n{softmax(arr_2d)}")
```

Output:

```
* --- 1D Array Example ---
Input array: [1. 2. 3.]
Softmax output: [0.09003057 0.24472847 0.66524096]

--- 2D Array Example ---
Input array:
[[1. 2. 3.]
 [4. 5. 6.]]
Softmax output:
[[0.09003057 0.24472847 0.66524096]
 [0.09003057 0.24472847 0.66524096]]
```

Task 06: Generating a NumPy array with 1000 rows and 5 columns, where each value is randomly sampled from a normal distribution. Display the first 5 rows of the generated data.

Program:

```
import numpy as np

np.random.seed(42) # for reproducibility
random_data_matrix = np.random.normal(size=(1000, 5))
print("First 5 rows of the generated 1000x5 array:")
print(random_data_matrix[:5, :])
```

Output:

```
• First 5 rows of the generated 1000x5 array:
[[ 0.49671415 -0.1382643  0.64768854  1.52302986 -0.23415337]
 [-0.23413696  1.57921282  0.76743473 -0.46947439  0.54256004]
 [-0.46341769 -0.46572975  0.24196227 -1.91328024 -1.72491783]
 [-0.56228753 -1.01283112  0.31424733 -0.90802408 -1.4123037 ]
 [ 1.46564877 -0.2257763  0.0675282 -1.42474819 -0.54438272]]
```

International Islamic University Islamabad
Faculty of Engineering and Technology
Department of Electrical Engineering

Artificial Intelligence & Machine Learning LAB

Lab No. 3

Performing Data Analysis and Visualizing Results Using Pandas and Matplotlib

Name: Bushra Nazir

Reg. No.: 1071-F22

S. No.	Criteria	Allocated Percentage	Level 1 (0%)	Level 2 (25%)	Level 3 (50%)	Level 4 (75%)	Level 5 (100%)	Marks Obtained
(A) PSYCHOMOTOR (To be judged in the lab during experiment)								
1	Practical Implementation OR Connectivity/Arrangement of Equipment	5	Absent	Degree of errors in applying procedural knowledge or Degree of errors in arrangement of equipment:				
			0	With several critical errors, Incomplete and Not Neat	With few errors, Incomplete and not Neat	With some errors, Complete but not Neat	Without errors, Complete and Neat	
				1.25	2.5	3.75	5	
2	Use of Equipment OR Use of Simulation/Programming Tool	2	Absent	Use of tools, equipment and materials with:				
			0	Limited Competence	Some Competence	Considerable Competence	Good Competence	
				0.5	1	1.5	2	
3	Safety (Optional)	0	Absent	Degree of reminders to follow safety procedure with:				
			0	Rarely Follow Safety Rules	Needs Reminders	Needs minor Reminders	Follows Safety Rules	
				0	0	0	0	
SUB-TOTAL (PT)		7	SUB-TOTAL MARKS OBTAINED (PO)					
(B) COGNITIVE (To be judged on the copy of experiment submitted)								
4	Data Record, Analysis and Evaluation OR Algorithm Design	1	Absent	Data record, Analysis and Evaluation or Designed Algorithm is:				
			0	Incorrect /Incomplete	Complete with some errors	Complete with few errors	Complete and Accurate	
				0.25	0.5	0.75	1	
5	Procedural Awareness (Optional)	0	Absent	Procedural awareness of the completed task is:				
			0	Inappropriate	Appropriate with some errors	Appropriate with few errors	Appropriate	
				0	0	0	0	
SUB-TOTAL (CT)		1	SUB-TOTAL MARKS OBTAINED (CO)					
(C) AFFECTIVE (To be judged in the lab during experiment)								
6	Level of Participation & Attitude to Achieve Individual/Group Goals	2	Absent	Degree of positive interaction as an individual or within a group:				
			0	Rare interaction	Some interaction	Acceptable interaction	Good interaction	
				0.5	1	1.5	2	
SUB-TOTAL (AT)		2	SUB-TOTAL MARKS OBTAINED (AO)					

Total Marks (PT + CT + AT) :

10

Instructor

Marks Obtained (PO + CO + AO) :